



# CONSIGNA DE TRABAJO

## CAPSTONE

**Curso:** Taller de Proyectos 1 – ISI

**Unidad:** Construcción del Producto Mínimo Viable

**Resultado de aprendizaje:**

Al finalizar la Unidad, el estudiante será capaz de **construir el producto mínimo viable bajo la guía del plan y el diseño elaborado.**

---



## PROPÓSITO DE LA CONSIGNA

El equipo desarrollará un **PMV funcional** basado en:

- Historias de usuario priorizadas
- Diseño arquitectónico (Arquitectura Hexagonal)
- Plan de desarrollo definido

El producto debe incluir:

- Front-end funcional
  - Back-end con lógica de negocio
  - Base de datos persistente
  - Integración con servicio externo (IA o API)
- 



## ORGANIZACIÓN DEL EQUIPO

### (Roles obligatorios)

| Rol                                   | Responsabilidades clave                                                                                      |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Arquitecto de Software                | Define estructura hexagonal, estándares, dependencias, decisiones técnicas<br><b>Chuchon Sotelo, Ernesto</b> |
| Desarrollador Front-End               | Implementa interfaces de usuario y consumo de APIs<br><b>Huachos Lopez, Madelyn</b>                          |
| Desarrollador Back-End                | Implementa lógica de negocio, casos de uso y controladores<br><b>Carhuamaca Vasquez, Diego</b>               |
| Desarrollador Base de Datos           | Diseña e implementa el modelo de datos y persistencia<br><b>Reinaga Cisneros, Hermiluz</b>                   |
| Desarrollador de Integración (IA/API) | Integra servicios externos (IA, APIs REST, etc.)                                                             |

---

# PARTE 1: GUÍA DE INSTALACIÓN DEL ENTORNO

El equipo debe documentar y ejecutar la instalación.

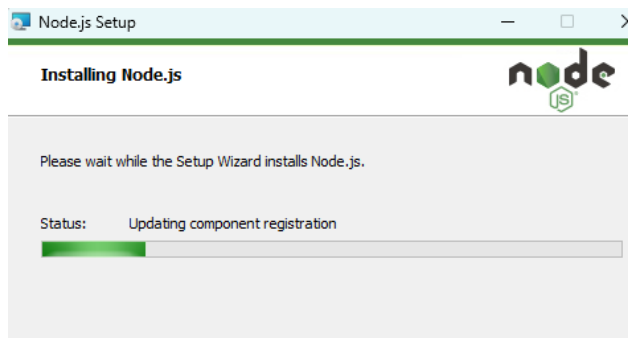
## 1. Front-End

Seleccionar framework (ejemplo sugerido):

- React Vite

**Pasos mínimos:**

1. Instalar Node.js



```
PS C:\Users\estudianteuc> npm --version
>>
11.19.0
PS C:\Users\estudianteuc> node --version
v24.21.0
PS C:\Users\estudianteuc> |
```

```

PS D:\frontend> npm create vite@latest mi-app -- --template react
npm notice run npx
npm notice run create-vite mi-app --template react
|
◇ Which linter to use?
  Oxlint
◇ Install with npm and start now?
  Yes
◇ Scaffolding project in D:\frontend\mi-app...
|
◇ Installing dependencies with npm...

added 24 packages, and audited 25 packages in 24s

9 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
|
◇ Starting dev server...
npm notice run mi-app@0.0.0 dev
npm notice run vite

```

2. Crear proyecto (npx create-react-app o equivalente)

3. Configurar estructura de carpetas
4. Instalar dependencias:
  - Axios o Fetch
  - Librerías UI (opcional)
5. Ejecutar proyecto local

## 2. Back-End

### 2.1 Tecnología seleccionada

Para el desarrollo del Back-End del sistema se utilizará Python 3.12 con FastAPI y Uvicorn. FastAPI será utilizado para implementar la API REST encargada de recibir las solicitudes provenientes del Front-End, ejecutar los casos de uso del sistema y comunicarse con la capa de persistencia y los servicios de inteligencia artificial.

Las principales tecnologías utilizadas son:

| Herramienta | Uso                                     |
|-------------|-----------------------------------------|
| Python 3.12 | Lenguaje principal del Back-End         |
| FastAPI     | Desarrollo de la API REST               |
| Uvicorn     | Servidor ASGI para ejecutar FastAPI     |
| Pydantic    | Validación de datos de entrada y salida |
| SQLAlchemy  | Comunicación con PostgreSQL             |
| Alembic     | Gestión de migraciones de base de datos |
| Postman     | Prueba de endpoints REST                |

El Back-End será desarrollado bajo una arquitectura hexagonal, separando la lógica de dominio, los casos de uso y la infraestructura.

## 2.2 Verificación de Python

Primero se verificó la instalación de Python

### Evidencia:

Figura 1. Verificación de la versión instalada de Python.



## 2.3 Creación de la carpeta del Back-End

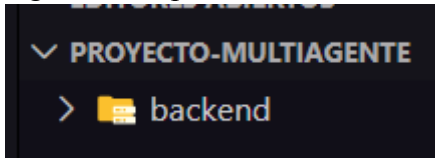
Se creó una carpeta independiente para almacenar el código correspondiente al Back-End del proyecto. Ejemplo de estructura inicial:

```
proyecto-multiagente/  
└─ backend/
```

Posteriormente, la carpeta fue abierta utilizando Visual Studio Code.

### Evidencia:

Figura 2. Carpeta del Back-End abierta en Visual Studio Code.



## 2.4 Creación del entorno virtual

Para mantener aisladas las dependencias del proyecto se creó un entorno virtual de Python. Desde la terminal, dentro de la carpeta `backend`, se ejecutó:

```
Shell  
python -m venv venv
```

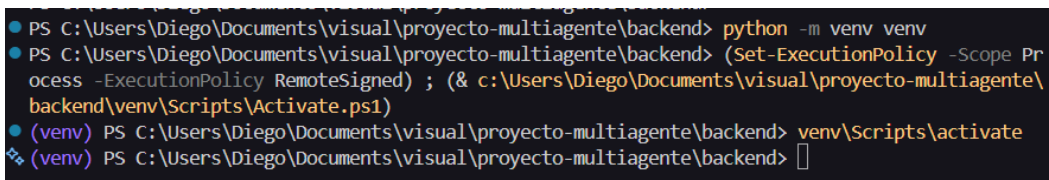
Posteriormente se activó el entorno virtual. En Windows:

```
Shell  
venv\Scripts\activate
```

Cuando el entorno se encuentra correctamente activado, la terminal muestra: `(venv)` al inicio de la línea de comandos.

### Evidencia:

Figura 3. Entorno virtual del Back-End activado.

A screenshot of a Windows PowerShell terminal window. It shows the following commands and output:  
1. `PS C:\Users\Diego\Documents\visual\proyecto-multiagente\backend> python -m venv venv`  
2. `PS C:\Users\Diego\Documents\visual\proyecto-multiagente\backend> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Diego\Documents\visual\proyecto-multiagente\backend\venv\Scripts\Activate.ps1)`  
3. `(venv) PS C:\Users\Diego\Documents\visual\proyecto-multiagente\backend> venv\Scripts\activate`  
4. `(venv) PS C:\Users\Diego\Documents\visual\proyecto-multiagente\backend>`  
The prompt changes from `PS` to `(venv)` after activation.

## 2.5 Instalación de dependencias

Con el entorno virtual activado, se instalaron las dependencias iniciales necesarias para desarrollar la API REST:

```
Shell  
pip install fastapi uvicorn pydantic
```

Para la integración posterior con PostgreSQL se consideran además:

```
Shell
pip install sqlalchemy psycopg alembic
```

Las dependencias instaladas pueden verificarse mediante:

```
Shell
pip list
```

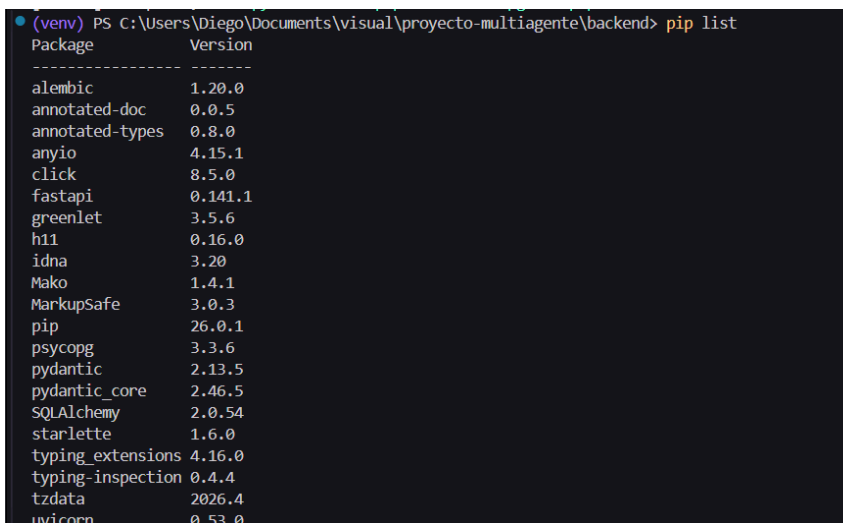
Asimismo, se generará el archivo de dependencias del proyecto mediante:

```
Shell
pip freeze > requirements.txt
```

El archivo `requirements.txt` permitirá que los demás integrantes puedan reproducir el mismo entorno de desarrollo.

### Evidencia:

Figura 4. Instalación y verificación de dependencias del Back-End.



```
(venv) PS C:\Users\Diego\Documents\visual\proyecto-multiagente\backend> pip list
Package            Version
-----
alembic             1.20.0
annotated-doc       0.0.5
annotated-types     0.8.0
anyio               4.15.1
click               8.5.0
fastapi             0.141.1
greenlet            3.5.6
h11                 0.16.0
idna                3.20
Mako                1.4.1
MarkupSafe          3.0.3
pip                 26.0.1
psycopg             3.3.6
pydantic            2.13.5
pydantic_core       2.46.5
SQLAlchemy          2.0.54
starlette           1.6.0
typing_extensions   4.16.0
typing-inspection   0.4.4
tzdata              2026.4
uvicorn             0.53.0
```

## 2.6 Creación de la estructura base

De acuerdo con la arquitectura hexagonal requerida para el proyecto, se creó una estructura inicial separada en las capas de dominio, aplicación e infraestructura. La consigna establece precisamente esta separación entre dominio, aplicación e infraestructura.

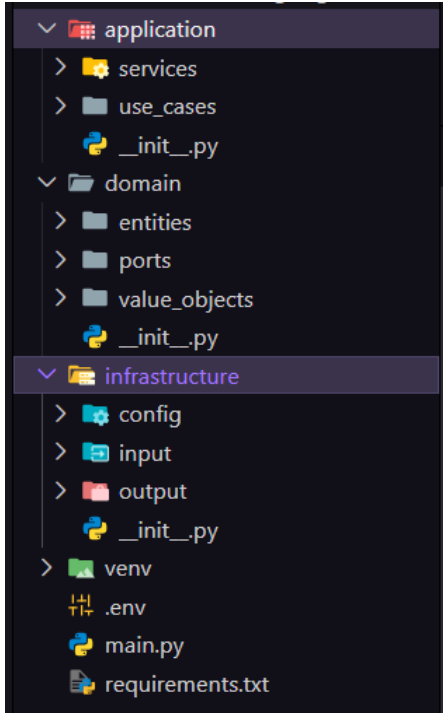
Cada capa tendrá la siguiente responsabilidad:

1. **Domain:** contiene las entidades, reglas de negocio e interfaces del sistema.
2. **Application:** contiene los casos de uso y servicios de aplicación.

3. **Infrastructure:** contiene los controladores REST, acceso a base de datos e integraciones externas.

### Evidencia:

Figura 5. Estructura inicial del Back-End bajo arquitectura hexagonal.



## 2.7 Configuración inicial de FastAPI

Se creó el archivo `main.py` para verificar el funcionamiento inicial del servidor FastAPI. Código utilizado:

```
Python
from fastapi import FastAPI

app = FastAPI(
    title="Sistema Multiagente para Priorización de Proyectos",
    version="1.0.0"
)

@app.get("/")
def inicio():
    return {
        "mensaje": "Backend del Sistema Multiagente funcionando correctamente"
    }
```

```
@app.get("/health")  
  
def health():  
    return {  
        "status": "ok"  
    }  
}
```

Este código permite verificar que la API se encuentra disponible antes de implementar las historias de usuario.

## 2.8 Ejecución del servidor

Para iniciar el servidor de desarrollo se ejecutó:

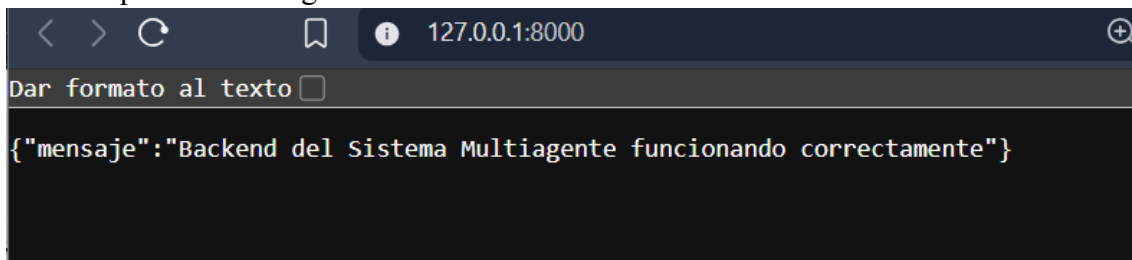
```
Shell  
uvicorn main:app --reload
```

Una vez iniciado correctamente, la API se encuentra disponible en: <http://127.0.0.1:8000>

### Evidencia:

```
(venv) PS C:\Users\Diego\Documents\visual\proyecto-multiagente\backend> uvicorn main:app --reload  
INFO: Will watch for changes in these directories: ['C:\\Users\\Diego\\Documents\\visual\\proyecto-multiagente\\backend']  
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)  
INFO: Started reloader process [10432] using StatReload  
INFO: Started server process [21200]  
INFO: Waiting for application startup.  
INFO: Application startup complete.  
INFO: 127.0.0.1:62750 - "GET / HTTP/1.1" 200 OK
```

Y otra captura del navegador:



```
< > ↻ ⓘ 127.0.0.1:8000  
Dar formato al texto ☐  
  
{"mensaje": "Backend del Sistema Multiagente funcionando correctamente"}
```

## 2.9 Documentación automática de la API

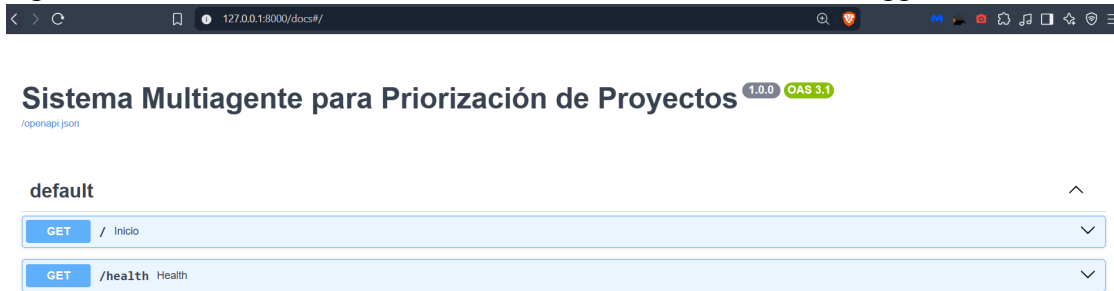
FastAPI genera automáticamente documentación interactiva mediante Swagger UI. Se accede desde: <http://127.0.0.1:8000/docs>

En esta interfaz se pueden visualizar y probar los endpoints disponibles. Inicialmente se encuentran disponibles:

4. GET /
5. GET /health

## Evidencia:

Figura 7. Documentación automática de la API REST mediante Swagger UI.



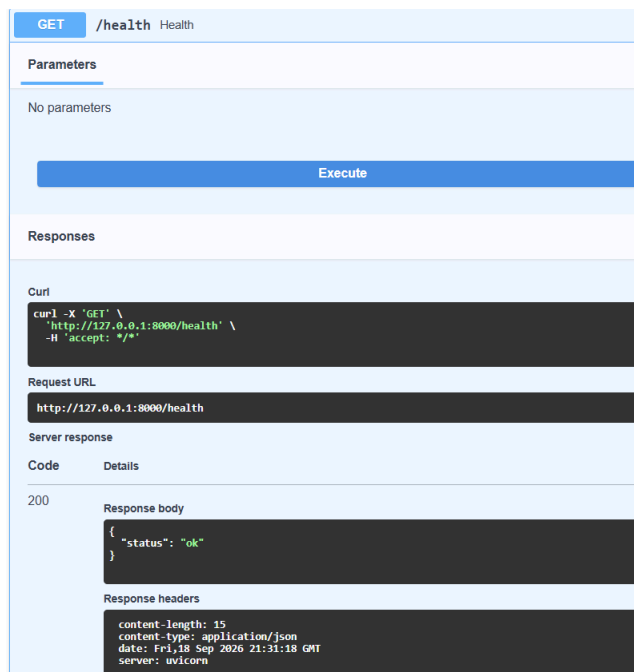
## 2.10 Prueba inicial del Back-End

Se realizó una prueba sobre el endpoint: GET /health

La respuesta confirma que el servicio Back-End se encuentra operativo y preparado para implementar los casos de uso. La consigna también solicita que la API REST sea probada mediante herramientas como Postman.

## Evidencia:

Figura 8. Prueba del endpoint GET /health.



## 2.11 Historia de usuario que se implementará

Como primera funcionalidad del Back-End se implementará la historia de usuario HU1.1, perteneciente al PMV1: *Como Planificador de Desarrollo Urbano, quiero cargar el expediente de mi proyecto una sola vez, para que el sistema lo reparta automáticamente entre los cinco agentes especialistas.*

Para esta historia se implementará inicialmente el endpoint: **POST /proyectos**

El endpoint recibirá los datos principales del proyecto, realizará su validación y enviará la información a la capa de aplicación para su registro.

se realizará en la siguiente fase, cuando se integren el Front-End y la base de datos.

### 3. Base de Datos

Tecnología sugerida:

- PostgreSQL / MySQL / MongoDB

**Pasos mínimos:**

#### 1. Instalar motor de base de datos

Se creó una cuenta en Neon (neon.tech) y un proyecto llamado siprim, con el servicio PostgreSQL activado

#### 2. Crear base de datos

Neon crea por defecto la base neondb. Desde la opción Databases → Add database se creó la base de datos siprim, con propietario neondb\_owner, de uso exclusivo para el sistema.

| Databases                                                                                                  |                       |                              |                                   |                            | <button>Add database</button> |
|------------------------------------------------------------------------------------------------------------|-----------------------|------------------------------|-----------------------------------|----------------------------|-------------------------------|
| All projects / siprim ↗ production <b>Default</b>                                                          |                       |                              |                                   |                            |                               |
| Manage the Postgres databases on this branch. For more information, see <a href="#">Manage databases</a> . |                       |                              |                                   |                            |                               |
| neondb                                                                                                     | Owner<br>neondb_owner | Created<br>5 minutes ago     | Last updated<br>5 minutes ago     | <button>Edit data</button> | ⋮                             |
| siprim                                                                                                     | Owner<br>neondb_owner | Created<br>A few seconds ago | Last updated<br>A few seconds ago | <button>Edit data</button> | ⋮                             |

### 3. Definir modelo (tablas o colecciones)

Para la historia de usuario HU1.1 se definió la tabla proyectos, que almacena el expediente básico de cada proyecto de inversión.

Untitled

Save

Primary

Active

siprim

⌚

✦

```

1 CREATE TABLE IF NOT EXISTS proyectos (
2     id SERIAL PRIMARY KEY,
3     nombre VARCHAR(200) NOT NULL,
4     descripcion TEXT NOT NULL,
5     ubicacion VARCHAR(300) NOT NULL,
6     presupuesto NUMERIC(18,2) NOT NULL CHECK (presupuesto > 0),
7     beneficiarios INTEGER NOT NULL CHECK (beneficiarios > 0),
8     tipo_proyecto VARCHAR(150) NOT NULL,
9     estado VARCHAR(30) NOT NULL DEFAULT 'REGISTRADO',
10    fecha_creacion TIMESTAMPTZ NOT NULL DEFAULT NOW()
11 );

```

Connected (1 query)

Run

Explain

Analyze

Statement executed successfully

Tables

All proj... / sip... ↗ pro... **Default**

siprim

Schema

public

Search...

+

proyectos

id serial

nombre varchar(200)

descripcion text

ubicacion varchar(300)

presupuesto numeric(18,2)

115ms

0 - 0 of 0

↺

↻

⋮

### 4. Configurar conexión con backend

Se configuró la dirección de la base de datos en el Back-End y una prueba de conexión confirmó que ambos quedaron conectados.

```

(venv) PS C:\Users\LENOVO\Downloads\SIPRIM-main\SIPRIM-main> python -m backend.infrastructure.output.database.init_db
Conectando a: postgresql+psycopg://neondb_owner:***@ep-polished-breeze-b6mlk084-pooler.c-2.sa-east-1.aws.neon.tech/siprim?sslmode=require
Conexion OK -> PostgreSQL 18.6 (6569466) on aarch64-unknown-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0, 64-bit
Tablas disponibles: ['proyectos']

```

### 5. Ejecutar pruebas CRUD

## CRUD\_POST

Prueba CREATE: registro de un proyecto mediante POST

http://127.0.0.1:8000/api/v1/proyectos

Server response

Code

Details

201

Response body

```
{
  "mensaje": "Proyecto registrado correctamente",
  "proyecto": {
    "id": 1,
    "nombre": "Mejoramiento de parque urbano",
    "descripcion": "Recuperacion de espacio publico en El Tambo",
    "ubicacion": "El Tambo - Huancayo",
    "presupuesto": 2500000,
    "beneficiarios": 5000,
    "tipo_proyecto": "Infraestructura urbana",
    "estado": "REGISTRADO",
    "fecha_creacion": "2026-09-24T17:34:12.824856Z"
  }
}
```

Download

Response headers

```
access-control-allow-credentials: true
content-length: 357
content-type: application/json
date: Thu, 24 Sep 2026 17:34:18 GMT
server: uvicorn
vary: Origin
```

Responses

Code

Description

Links

201

Successful Response

No links

sigrim

production

Connect

Overview

Branches

+ Add record

108ms

< 1 - 1 of 1 >

⌂

⋮

| <input type="checkbox"/> | id serial | nombre v...  | descripcion text   | ubicacio...   | presupuest... | beneficiar... | tipo_pro...      | estado varchar(10) | fecha_creacion timestamp... | + |
|--------------------------|-----------|--------------|--------------------|---------------|---------------|---------------|------------------|--------------------|-----------------------------|---|
| <input type="checkbox"/> | 1         | Mejoramen... | Recuperacion de... | El Tambo ~... | 2500000.00    | 5000          | Infraestructu... | REGISTRADO         | 2026-09-24 17:34:12.02...   |   |

## CRUD GET

Prueba READ: consulta del proyecto mediante GET

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/api/v1/proyectos/1' \
  -H 'accept: */*'
```

Request URL

http://127.0.0.1:8000/api/v1/proyectos/1

Server response

Code

Details

200

Response body

```
{
  "id": 1,
  "nombre": "Mejoramiento de parque urbano",
  "descripcion": "Recuperacion de espacio publico en El Tambo",
  "ubicacion": "El Tambo - Huancayo",
  "presupuesto": 2500000,
  "beneficiarios": 5000,
  "tipo_proyecto": "Infraestructura urbana",
  "estado": "REGISTRADO",
  "fecha_creacion": "2026-09-24T17:34:12.824856Z"
}
```

Download

Response headers

```
content-length: 298
content-type: application/json
date: Thu, 24 Sep 2026 17:44:56 GMT
server: uvicorn
vary: Origin
```

Responses

Code

Description

Links

200

Successful Response

No links

## CRUD PUT

Prueba UPDATE: actualización del proyecto mediante PUT

Server response

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "mensaje": "Proyecto actualizado correctamente",   "proyecto": {     "id": 1,     "nombre": "Mejoramiento de parque urbano - Etapa 2",     "descripcion": "Recuperacion de espacio publico en El Tambo",     "ubicacion": "El Tambo - Huancayo",     "presupuesto": 3000000,     "beneficiarios": 5000,     "tipo_proyecto": "Infraestructura urbana",     "estado": "REGISTRADO",     "fecha_creacion": "2026-09-24T17:34:12.024050Z"   } }</pre> <p>Response headers</p> <pre>access-control-allow-credentials: true content-length: 368 content-type: application/json date: Thu, 24 Sep 2026 17:48:47 GMT server: uvicorn vary: Origin</pre> |

Responses

| Code | Description         | Links    |
|------|---------------------|----------|
| 200  | Successful Response | No links |

## CRUD DELETE

Prueba DELETE: eliminación del proyecto mediante

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/api/v1/proyectos/1' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/api/v1/proyectos/1
```

Server response

| Code | Details                                                                                                                                    |
|------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 204  | <p>Response headers</p> <pre>access-control-allow-credentials: true date: Thu, 24 Sep 2026 17:49:52 GMT server: uvicorn vary: Origin</pre> |

Responses

| Code | Description         | Links    |
|------|---------------------|----------|
| 204  | Successful Response | No links |

## 4. Integración con Servicios Externos (IA/API)

Ejemplos:

- API de IA (chat, recomendación, clasificación)
- API externa (clima, pagos, geolocalización)

#### **Pasos mínimos:**

1. Crear cuenta en proveedor de API
2. Obtener API Key
3. Configurar variables de entorno
4. Implementar cliente HTTP
5. Probar integración desde backend

---

## **PARTE 2: IMPLEMENTACIÓN BAJO ARQUITECTURA HEXAGONAL**



### **Estructura obligatoria**

```
/domain
├── entities
├── valueObjects
└── ports (interfaces)

/application
├── useCases
└── services

/infrastructure
├── adapters
│   ├── input (controllers)
│   └── output (DB, APIs)
```

---



### **Programación requerida**

#### **1. Dominio**

- Entidades principales del sistema
- Reglas de negocio
- Interfaces (puertos)

#### **2. Aplicación**

- Casos de uso por historia de usuario
- Servicios de aplicación

#### **3. Infraestructura**

- Controladores REST
  - Repositorios
  - Integración con APIs externas
- 

## **Relación con Historias de Usuario**

Cada historia debe tener:

- Endpoint asociado
  - Caso de uso implementado
  - Persistencia (si aplica)
  - Vista (front-end)
-



# PLAN DE MONITOREO (3 SESIONES)

---



## SESIÓN 1: Configuración e Instalación

**Objetivo:** Entorno listo y arquitectura base creada

**Actividades:**

- Instalación completa (FE, BE, BD)
- Estructura hexagonal inicial
- Configuración repositorio (GitHub)

**Evidencias:**

- Proyecto ejecutándose localmente
- Documento de instalación
- Estructura del proyecto

**Monitoreo docente:**

- Validación de entornos
  - Revisión de arquitectura
  - Retroalimentación técnica
- 



## SESIÓN 2: Implementación de Casos de Uso

**Objetivo:** Desarrollo del núcleo del PMV

**Actividades:**

- Implementar mínimo 2–3 historias de usuario
- Desarrollo de:
  - Entidades
  - Casos de uso
  - Endpoints
- Conexión con base de datos

**Evidencias:**

- APIs funcionales
- Código estructurado en capas

- Pruebas básicas

**Monitoreo docente:**

- Revisión de cumplimiento de arquitectura hexagonal
  - Evaluación de separación de responsabilidades
  - Observación de roles
- 



## **SESIÓN 3: Integración y Validación del PMV**

**Objetivo:** Producto funcional integrado

**Actividades:**

- Integración Front-End ↔ Back-End
- Integración con API externa o IA
- Pruebas funcionales
- Ajustes finales

**Evidencias:**

- PMV funcionando end-to-end
- Demo del sistema
- Documentación técnica

**Monitoreo docente:**

- Validación del flujo completo
  - Evaluación del valor del PMV
  - Retroalimentación final
- 



## **ENTREGABLES FINALES**

1. Repositorio GitHub organizado
  2. Documento de instalación
  3. Código fuente estructurado (hexagonal)
  4. PMV funcional
  5. Video demo (opcional)
-



# CRITERIOS CLAVE DE EVALUACIÓN

- Correcta implementación de arquitectura hexagonal
  - Coherencia con historias de usuario
  - Integración funcional del sistema
  - Uso adecuado de roles
  - Calidad del código
-